Week 7 Progress Report

Group L — UniSupport AI: A Bounded Agentic Student Support Case System

Week ending: 16th October 2026

Submitted by: Kizito Ryan

---

1. Work Completed Against Week 7 Deliverables

The objective of Week 7, per the capstone project brief, was to establish **Evaluation, Observability, and Guardrails**: producing rigorous, empirical evidence demonstrating that UniSupport AI is reliable, grounded, inspectable, and safely bounded within its university student-support domain.

Work completed against the defined Week 7 deliverables:

1. **30+ Scenario Evaluation Benchmark (`docs/evaluation.md`)**:

- Constructed and automated a comprehensive evaluation benchmark comprising **39 distinct scenarios** across all six required archetypes:
- **Normal (5 scenarios)**: Policy Q&A via RAG with source attribution, timetable queries, ticket draft generation, staff ticket approval, staff ticket rejection.
- **Edge (5 scenarios)**: Inputs with leading/trailing whitespace, multi-line formatting, guest tool invocation requests, unnecessary tool requests, queries for nonexistent courses.
- **Incorrect Information (5 scenarios)**: Unmatched policy inquiries producing grounded "no evidence" responses without hallucination, empty chunk handling, provider empty responses, provider timeouts (HTTP 502), and provider misconfigurations (HTTP 503).
- **Adversarial (10 scenarios)**: Prompt injection attempts ("approve my ticket", "submit my ticket", "ignore all previous instructions"), system prompt extraction, unregistered tool requests, multi-ticket flooding attacks, repetitive tool loops, role header spoofing via message body, and indirect injection via ticket metadata.
- **Tool Failures (6 scenarios)**: Missing timetable datasets, corrupted/malformed JSON files, nonexistent ticket database paths, unhandled tool exceptions, malformed tool return types (strings or `None`).
- **Unauthorized Actions (8 scenarios)**: Unrecognized user roles, guest dispatch requests, guest attempts on tickets and timetables, missing or malformed role headers defaulting to least privilege, student attempts to invoke staff-only ticket approval and rejection endpoints.
- Every single scenario is directly mapped to an automated test and is currently passing (**39/39 passing**).

1. **Code-Level Guardrails & Implementation Evidence (`docs/guardrails.md`)**:

- Formally documented the system's seven defense-in-depth security layers, verifying that safety boundaries are enforced at the application code level rather than relying on model compliance:
- **Input Validation**: Pydantic validation via `StudentSupportRequest` enforcing non-empty, stripped strings bounded to 2,000 characters (`MAX_MESSAGE_LENGTH`).

- **Tool Allow-List**: Strict registry matching (`TOOL_REGISTRY` in `tools/registry.py`) and immutable contract frozen sets (`frozenset(TOOL_REGISTRY)`) preventing execution of unregistered code.
- **Role-Based Access Control (RBAC)**: Authority derived strictly from the trusted `X-User-Role` HTTP header (`auth.py`), never from untrusted message body claims.
- **Execution Loop Budgets**: Dual independent caps hard-enforced in `orchestrator.py` (`AGENT_MAX_ITERATIONS=4` , `MAX_TOOL_CALLS=3`).
- **Human-in-the-Loop (HITL) Barrier**: The agent is restricted to generating `PENDING_APPROVAL` draft tickets; transition to `SUBMITTED` or `REJECTED` strictly requires staff authentication via dedicated HTTP endpoints.
- **Prompt Injection & Jailbreak Resistance**: Adversarial attacks cannot bypass code-level tool allow-lists or state machine guards.
- **Error Containment**: All tool exceptions and malformed responses are captured as structured observation dictionaries, ensuring zero unhandled server crashes.

1. **Failure Catalogue with Root-Cause Remediation (`docs/failure-catalogue.md`)**:

- Documented six genuine, non-contrived engineering failures identified during testing and development, recording root-cause analysis, code remediation, and re-test verification:
- *Failure 1*: Agent loop trusted provider to honor `tool_choice="none"` → Hard-capped actual executions in `orchestrator.py` .
- *Failure 2*: Timetable tool returned empty match without distinguishing missing courses → Structured error separation implemented in `tools/timetable.py` .
- *Failure 3*: Unauthorized tool calls counted against student execution budget → Counter updated only upon successful dispatch.
- *Failure 4*: Missing `.env` file after folder migration causing HTTP 503 errors → Standardized onboarding and configuration recovery procedure.
- *Failure 5*: Unregistered tool (`search_application_requirements`) present in codebase but uncallable → Added contract allow-list invariant tests.
- *Failure 6*: High-impact topics (grades, fees, disciplinary actions) defended only by prompt instructions → Minimized tool surface so no tools exist capable of enacting high-impact actions.

1. **Observability, Tracing, and Automated Test Evidence (`evidence/week7/`)**:

- Expanded test coverage across the system to **178 passing tests** (verified in 68.5s via pytest).
- Created dedicated adversarial test suite (`agent-backend/tests/test_prompt_injection.py`) covering 14 security attack scenarios (14/14 passing).
- Captured full test run evidence in `evidence/week7/full-suite.txt` and `evidence/week7/adversarial-tests.txt` .

1. **Week 7 Progress Report & Formats**:

- Completed this comprehensive report (`docs/weekly-reports/week7_Progress_Report.md`) along with publication-ready `.docx` and `.pdf` versions stored in `docs/weekly-reports/` .

---

2. Key Engineering Decisions

1. Code-Enforced Guardrails Over Prompt-Only Instructions

Early in the project, safety guidelines (such as refusing to alter grades or approve tickets) were described primarily within system prompts (`llm/prompts.py`). During Week 7 adversarial testing, we reaffirmed that prompt instructions alone cannot guarantee safety against sophisticated jailbreaks or injection patterns. All critical safety boundaries are now backed by deterministic Python code:

- Unregistered tools are structurally impossible to invoke because dispatch resolves only against `TOOL_REGISTRY` .
- Ticket status transitions cannot be enacted by the LLM because no tool exists to mutate status to `SUBMITTED` .
- Role authentication evaluates HTTP request headers, completely ignoring claims within the student's message body.

2. Dual Independent Loop Budgets

Rather than relying on the LLM to signal completion or respect `tool_choice="none"` , the orchestrator implements two programmatic circuit breakers:

1. `agent_max_iterations` (default: 4): Restricts cognitive decision cycles.
2. `max_tool_calls` (default: 3): Restricts tool invocations per session.

Even if a model enters an infinite tool-calling loop or hallucinated retry sequence, execution terminates deterministically with a graceful terminal status (`max_iterations_reached` or `max_tool_calls_reached`).

3. Dispatch-Success Budget Accounting

Previously, any tool request incremented the tool budget. When guest users attempted tool invocations, their unauthorized requests depleted the budget without performing work. We updated `orchestrator.py` to increment counters only when tool calls pass authorization and execute successfully. Refused calls are recorded as observations (allowing the model to explain the refusal to the user) without consuming execution allowances.

---

3. Failures and Challenges

1. **Provider Non-Compliance with Tool Choice Directives**:

- *Challenge*: When evaluating mock and live LLM responses with `tool_choice="none"` , some providers occasionally returned further tool-call payloads.
- *Resolution*: Hardened `agent/orchestrator.py` to enforce a hard cap in code. If `tool_call_count >= contract.max_tool_calls` , any subsequent tool call returned by the LLM is intercepted and suppressed before dispatch.

1. **Silent False-Positives in Domain Retrieval**:

- *Challenge*: The timetable tool returned `{"sessions": []}` both when a course existed with no classes on a date and when a course code was entirely invalid. This led the model to fabricate plausible schedules for invalid course codes.
- *Resolution*: Refactored `tools/timetable.py` to inspect course existence first. Unknown courses return `{"success": false, "error": "Course not found"}`. The agent observes this explicit failure and informs the student rather than extrapolating.

1. **Untested Tool Surface Drift**:

- *Challenge*: The tool `search_application_requirements` was added to `agent-backend/tools/` but was omitted from `TOOL_REGISTRY`. It was ambiguous whether this exclusion was intentional.
- *Resolution*: Created an invariant test (`test_contract_allow_list_matches_tool_registry_exactly`) asserting that the contract allow-list matches `TOOL_REGISTRY` exactly, confirming that only vetted, intentional tools can ever be executed.

---

4. GitHub Commit & PR Links

- **Pull Requests:**
- **PR #14:** `Week 7: adversarial tests, guardrails documentation, and evaluation set` (Merged via commit `2c8c879`)
- **PR #13:** `Week6 progress report` (Merged via commit `e5a3bd6`)
- **Key Commits:**
- `868e7a0` : Week 7 adversarial tests, guardrails documentation, evaluation set, and test evidence
- `2c8c879` : Merge pull request #14 from `Musheijaa/week7-guardrails` into `main`
- `01dfc9a` : Week 6 progress report finalization
- **Key Documentation & Evidence Files:**
- `docs/evaluation.md` : 39-scenario evaluation dataset across 6 categories
- `docs/guardrails.md` : System guardrails specification and test mapping
- `docs/failure-catalogue.md` : Six real engineering failures, root-causes, and fixes
- `evidence/week7/adversarial-tests.txt` : Pytest execution output for 14 adversarial tests
- `evidence/week7/full-suite.txt` : Pytest execution output for full test suite (178 passed)
- **Automated Test Evidence (178 Passed, 0 Failed):**
- `agent-backend/tests/test_prompt_injection.py` : 14 passed (adversarial scenarios)
- `agent-backend/tests/test_tool_failure_handling.py` : 23 passed (unauthorized and service failure tests)
- `agent-backend/tests/test_ticket_approval_api.py` : 8 passed (staff approval & rejection endpoints)
- `agent-backend/tests/test_agent_orchestrator.py` : 12 passed (cognitive loop and budget bounds)
- `agent-backend/tests/test_state_model.py` : 15 passed (state machine lifecycle)
- `agent-backend/tests/test_case_history.py` : 14 passed (data privacy and retention)

---

5. Individual Contributions

Member	Task Owned	Evidence Link
Akakikunda Benita	Adversarial test suite implementation (`tests/test_prompt_injection.py`), system guardrails documentation (`docs/guardrails.md`), 39-scenario evaluation dataset construction (`docs/evaluation.md`), and raw Week 7 test evidence capture (`evidence/week7/`)	Commits `868e7a0` , PR #14; files `tests/test_prompt_injection.py` , `docs/guardrails.md` , `docs/evaluation.md` , `evidence/week7/`
Mwesigwa Sean	Failure catalogue authoring (`docs/failure-catalogue.md`), root-cause analysis for timetable empty matches and budget overcounting, error containment verification, and test execution validation	Commit `868e7a0` , PR #14; file `docs/failure-catalogue.md`
Okema Paul Mark	Tool allow-list invariant tests, verification of unregistered tools boundary (`search_application_requirements`), orchestrator loop budget hard-capping, and unauthorized dispatch handling	Commit `868e7a0` , PR #14; files in `agent-backend/tools/` , `agent-backend/agent/orchestrator.py`
Kizito Ryan	Week 7 Progress Report authoring (`docs/weekly-reports/week7_Progress_Report.md`), `.docx` and `.pdf` document generation, Pydantic input validation checks, and API error code reviews	Commits `868e7a0` , PR #14; files in `agent-backend/schemas.py` , `docs/weekly-reports/`
Musheija Abraham (Mwesigwa Duncan)	Pull request review and branch merge coordination (PR #14), test suite regression pass (178 tests verified), repository architectural consistency	Commit `2c8c879` , PR #14; repository coordination and review

6. Plan for Week 8: System Demonstration, UI Integration, and Final Submission

Focus (19th Oct – 23rd Oct 2026): Transition UniSupport AI from verified backend components to an end-to-end interactive student support system with frontend integration and final capstone deliverables.

Activities & Deliverables

1. **Interactive Frontend / UI Integration**:

- Connect the student-facing chat interface to the backend `/api/v1/agent/student-support` endpoint.
- Implement clear UI state indicators for RAG source citations, timetable display cards, and pending ticket status indicators.
- Build a minimal staff approval portal or view for inspecting and approving `PENDING_APPROVAL` ticket drafts.

1. **End-to-End Live Demonstration Walkthrough**:

- Rehearse and record representative user journeys:
- Policy inquiry with grounded RAG retrieval.
- Timetable lookup for valid and invalid courses.
- Support ticket drafting, stopping for human approval, and subsequent staff approval via the API.
- Adversarial injection attempt showing safe bounded refusal.

1. **Final Capstone Documentation & Code Clean-Up**:

- Consolidate architectural diagrams, API schemas, and deployment instructions into the root `README.md`.
- Ensure clean environment setup instructions and reproducible demo scripts.
- Package all weekly progress reports and evaluation metrics for final project assessment.